Watching Over the Reserve: A Fog-to-AWS Wildlife Habitat Monitoring Pipeline

Hrishikesh Sajeev
Student ID: X25132377
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25132377@student.ncirl.ie

***Abstract*—Rangers patrolling a reserve on a fixed schedule learn about a poaching incident, a drying waterhole, or an unusual movement spike only once they reach that spot, not when it starts. This paper presents a continuous two-reserve habitat-monitoring pipeline. Five sensor types per reserve feed a fog node that windows and aggregates readings and evaluates threshold alerts locally, forwarding only compact summaries to the cloud. A scalable serverless backend on Amazon SQS, AWS Lambda, and DynamoDB ingests those summaries and serves a live dashboard through Amazon API Gateway, with the static frontend hosted on Amazon S3. Each layer is justified against alternatives, and the system is deployed to a real AWS account rather than a local emulator. An automated suite of 82 tests passes, and end-to-end verification against the live deployment confirmed every pipeline health check reporting healthy, fresh readings arriving within seconds, and both reserves' metrics and alerts rendering correctly on the deployed dashboard. A pre-deployment audit additionally caught three defects a local emulator had masked.**

***Keywords*—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, wildlife conservation**

I. Introduction

A ranger relying on a fixed patrol schedule finds out about a poaching incident, a waterhole running dry, or an unusual surge in animal movement only once the patrol happens to reach that part of the reserve, not when the event actually starts. Continuous sensing changes that trade-off: a reserve instrumented with always-on acoustic, motion, and environmental sensors can raise an alert the moment a threshold is crossed, without waiting for a human to walk past.

Fog computing extends that idea to where the data is produced. Bonomi et al. describe fog computing as a platform distinguished by low latency, wide geographic distribution, and support for a large number of edge nodes [1], properties that matter directly here, since a reserve's sensor network is exactly the kind of geographically spread, intermittently connected deployment the definition has in mind. The National Institute of Standards and Technology frames cloud computing around five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [2]. Those characteristics describe the backend well but say nothing about what happens at the reserve itself, which is where a fog node earns its place in this design: it sits with the sensors, watches a rolling window of their readings, and is the one that decides whether a threshold such as a spike in acoustic activity near a waterhole has been crossed. By the time anything reaches the cloud backend, an entire window of individual readings has already been reduced to a handful of summary figures plus whatever alerts fired, so the elastic, on-demand side of the system in section II never has to ingest the raw sensor stream at all.

Three things had to come together for this paper's pipeline to count as done. The sensors and the fog node needed to actually buffer, aggregate, and evaluate five kinds of reading across two reserves at whatever sampling and dispatch rate each was told to run at, not a fixed rate baked into the code. The backend needed to turn that fog output into a dashboard capable of putting both reserves next to each other, using managed queueing and function-as-a-service rather than a server this project would have to keep patched and running itself. And none of that counted for anything unless it was deployed to a public cloud and checked against the deployment directly, not left as something the source code merely claimed to do.

The rest of the paper follows this structure. Section II lays out the architecture adopted at every layer, together with the alternatives that were weighed and set aside, and finishes with the security posture the deployed system settles into. Section III turns to implementation: the software stack and libraries in use, the automated test suite, the continuous-integration setup, the AWS deployment itself with the defects that surfaced once it went live, and the evidence measured directly from the running system. Section IV closes with a critical assessment of the project and an outline of future work.

II. System Architecture and Design

A. Sensor and Fog Layer

Two simulated reserves, reserve-a and reserve-b, carry five sensor types apiece: camera-trap motion-event counts, an anomalous acoustic-signature level used as a proxy for gunshot or chainsaw detection, waterhole level, ambient temperature, and soil moisture. Deploying low-cost acoustic sensors to flag gunshot-like signatures in a protected reserve is an established technique, not a hypothetical one: Prince et al. describe exactly this kind of algorithm running on open-source hardware in a tropical forest reserve [3], and the acoustic-signature threshold

TABLE I
ALERT THRESHOLDS (EVALUATED ON THE WINDOW AGGREGATE)

Sensor Type	Rule	Alert Key
Acoustic level	avg $>$ 75	Poaching risk
Waterhole level	avg $<$ 20	Drought stress
Motion-event count	max $>$ 30	Unusual activity
Soil moisture	avg $<$ 10	Habitat dryness

rule follows the same intent at a much smaller scale. Every sensor process reads two independently configurable settings, one for the sampling interval and one for the dispatch interval, so sampling frequency and dispatch rate are decoupled knobs, not one value aliased to both. Each dispatch call posts a small JSON object to the fog node carrying the sensor type, site identifier, measurement unit, and the batch of timestamped readings accumulated since the previous dispatch, typically five readings at the two- and ten-second sampling and dispatch defaults; with only a handful of short numeric fields per reading, the resulting body is a few hundred bytes at most, well under a kilobyte.

Every reading is buffered in a single atomic reference holding the whole reading map, mutated exclusively through an atomic compare-and-swap update: a lock-free, whole-structure copy-on-write retry loop, avoiding both a lock and a per-key concurrent map. Each ingest builds a brand-new immutable snapshot of the map and hands it to the atomic update, which automatically retries if a competing thread's update won the race in between; draining the buffer is a single atomic swap that hands the whole retiring snapshot to the caller while resetting the live reference to empty. This compare-and-swap retry pattern for concurrent data structures is a well-studied technique, not an ad hoc choice: Michael and Scott's classic non-blocking queue algorithms establish the same underlying idea of retrying a compare-and-swap instead of blocking a competing thread [4]. The trade-off, acceptable at this project's scale (five sensor types across two reserves, at most ten live keys), is that every ingest copies the whole snapshot; a 16-thread concurrent-ingest test proves no reading is ever lost to a missed retry.

On a fixed timer decoupled from buffer occupancy, the window cycle drains the buffer and reduces each surviving key to five figures (count, minimum, maximum, average, and the last-in-order latest reading) before the alert stage checks that figure set against Table I's threshold rules; the batch publisher then groups every window's aggregate-plus-alerts payload into batched sends capped at ten entries apiece, rather than one send per aggregate. If that outbound send to the queue fails for any reason, the window cycle logs the failure and moves directly on to the next window rather than retrying the send or buffering the lost window's aggregates locally for a later attempt, an accepted trade-off at the data volumes this deployment handles, where a single dropped window is a gap in the log rather than a record an operator would need to recover.

HTTP routes on the fog node (health, thresholds, and ingest) are discovered through Java reflection: every method marked with a routing annotation on the gateway object is registered automatically at startup by scanning the class's declared methods, with no hand-written route table to keep in sync. Reflection carries a documented runtime cost in exchange for that flexibility, a trade-off empirically characterised by Li et al. in their large-scale study of how Java applications actually use reflection [5]; at the request volumes this fog node handles, that cost is immaterial next to the benefit of never letting a route table drift out of sync with its handler methods.

B. Scalable Backend Layer

Nothing in the backend runs on a server this project provisions or patches itself: every piece of it is a managed AWS primitive. wcm-reserve-agg, a standard Amazon SQS queue, sits between the fog node's dispatch rate and however fast the backend happens to be processing at a given moment. Messages arriving on that queue automatically invoke wcm-processor, an AWS Lambda function, via an event-source mapping, and each invocation writes one aggregated window into the wcm-readings DynamoDB table. Its partition key is the sensor type, but the sort key is not just the window-end time on its own: it is the window-end time joined with the site, and the reason for pairing them only becomes clear once both reserves are reporting at the same time. If both reserves run a soil-moisture sensor and their flush cycles happen to close on the same window-end time, a sort key built from the window-end time alone would give both aggregates an identical partition and sort combination, so the second write would simply replace the first with no error surfaced anywhere: one reserve's reading for that window would just be gone. Folding the site into the sort key removes the ambiguity at the source, giving each reserve a separate row for the same window instead of requiring a global secondary index to tell them apart after the fact. The underlying pattern (partition on a coarse attribute, disambiguate within it via the sort key) is the same partition-and-replicate design DeCandia et al. describe as the basis for Amazon's internal Dynamo store, the system DynamoDB itself descends from [6].

Cold-start latency is the critique most often levelled at Lambda-based designs: without an already-warm execution environment, the runtime spends time initialising before the handler body even starts, and this delay has been measured directly in a large-scale production serverless deployment [7]. Two properties of this particular workload keep that latency from mattering in practice. The reading pipeline into wcm-processor is queue-triggered, so a slow cold start there simply delays how quickly a message is drained, not how quickly a browser renders anything: there is no request waiting on it. The dashboard Lambda sidesteps the issue differently: constant polling from the live dashboard keeps its container active often enough that a true cold start is uncommon once the dashboard has been polling for a while.

C. A Switch-Based Dispatch Table for the Lambda Deployment

The dashboard-facing Lambda, wcm-dashboard-api, sits behind an Amazon API Gateway REST API and answers every dashboard route: reserves, readings, thresholds, health, and backend statistics. Locally, the equivalent routes are discovered the same reflection-driven way the fog node's routes are, by scanning annotated methods over a running Java HTTP server. That server only exists for the lifetime of a running process, and a Lambda invocation never has one: there is no listening socket and no request object for reflection to bind a handler to. Carrying the reflection-based router into the deployment as-is was considered and set aside for exactly that reason: doing so would have meant either an adapter translating API Gateway's event shape into a fake request object, or reimplementing the reflection scan against a different request type entirely, for a set of routes (five fixed, static paths with no path parameters to extract) that neither needs. The deployed dashboard function instead answers each route by matching the request method and path directly, calling into the same repository, pipeline-check, and threshold classes the local routes already use, so the DynamoDB, SQS, and Lambda-metadata logic is never duplicated between the two entry points.

Four different things feed wcm-dashboard-api before it can answer a single request: readings and reserve comparisons come from the DynamoDB table, backlog depth comes from the SQS queue's own attributes, whether ingestion is still running comes from wcm-processor's Lambda metadata, and fog-side status comes from the fog node's /health and /thresholds endpoints, reached over the Elastic IP its EC2 instance is bound to. That last one carries a practical caveat worth naming directly: reaching a bare IP address on port 8000 over unencrypted HTTP, rather than a named domain on 443, is precisely the pattern that corporate and campus firewalls are configured to flag and drop, so putting the dashboard itself behind that same IP would have made the whole page unreachable from exactly the kind of network its intended viewers are likely sitting on. Its static files skip all of that and sit in an Amazon S3 bucket instead, with the dashboard script resolving its own API base address at startup by fetching a small configuration resource generated fresh at each deploy, rather than a build-time text substitution, an HTML meta tag, or a separate configuration script; it is left as an empty string for local development, where the same fog-node-adjacent host serves both origins.

D. Deployment Topology

The topology in Fig. 1 puts the two Lambda functions and everything downstream of them behind managed AWS services, leaving only the sensor and fog containers on infrastructure this project has to operate directly. Those containers sit on a single EC2 instance with no key pair issued for it at all (Session Manager is the only way in) and a security group that blocks every port except the fog node's own 8000. From there the top half of the figure traces the ingest side left to right: sensors dispatch to the fog node, the fog node batches to SQS, SQS triggers wcm-processor, and wcm-processor writes into DynamoDB. The bottom half traces the query side: the browser loads the dashboard from S3 and, separately, calls API Gateway directly, which routes to wcm-dashboard-api. That second Lambda is the only component reading back across both halves, pulling from DynamoDB and the three other sources named above to answer every request the dashboard makes.

E. Critical Analysis of Alternative Architectures

Several alternative designs were considered and either rejected or adopted for a specific, statable reason. The lock-free copy-on-write buffer was itself a deliberate departure from a plain synchronized map or a per-key concurrent map: a whole-structure compare-and-swap retry is only correct here because the request volume at this scale never turns the snapshot-copy cost into a bottleneck, a trade-off that would not hold at a much larger request volume, where a per-key lock or a concurrent map would avoid copying the whole snapshot on every call.

A hand-written route table for the fog node's HTTP layer was considered in place of the reflection-based route scan, and rejected because it would have meant maintaining a second, parallel list of path-to-handler mappings that could silently drift out of sync with the actual annotated methods as endpoints were added during development, a class of bug the reflection-driven approach makes structurally impossible, since the routing table is derived from the method metadata itself rather than copied from it by hand.

Apache Kafka was considered in place of SQS for the fog-to-processor link, and set aside. Kafka earns its complexity when several distinct consumer groups must independently rewind and re-read the same topic at their own pace, or when partition-level ordering has to survive a consumer restart; wcm-processor is the sole reader of these messages, and DynamoDB already orders each item by its window-end sort key once written, so nothing downstream would ever call on a stream-replay guarantee. SQS needs no partitions to provision and no broker cluster to size, and its event-source mapping wires directly into Lambda without a consumer-group client to configure, the simpler shape for a single, always-on consumer.

A single Lambda function handling both queue-triggered ingestion and API-Gateway-fronted query serving was considered in place of the two separate functions actually deployed, and rejected. wcm-processor's load tracks however many aggregate messages are waiting in the SQS queue at a given moment, while wcm-dashboard-api is invoked one-to-one by dashboard requests and must return within a much shorter window; merging the two under one reserved-concurrency limit and one timeout setting would mean a surge on either side eats into capacity the other needs: a queue backlog could leave dashboard requests waiting behind a function busy draining messages, and a run of slow downstream reads on the query side could just as easily leave the queue undrained.

Finally, converting the fog node and sensors themselves to a scheduled-Lambda model, so that the entire system, not only the backend, would be serverless, was considered

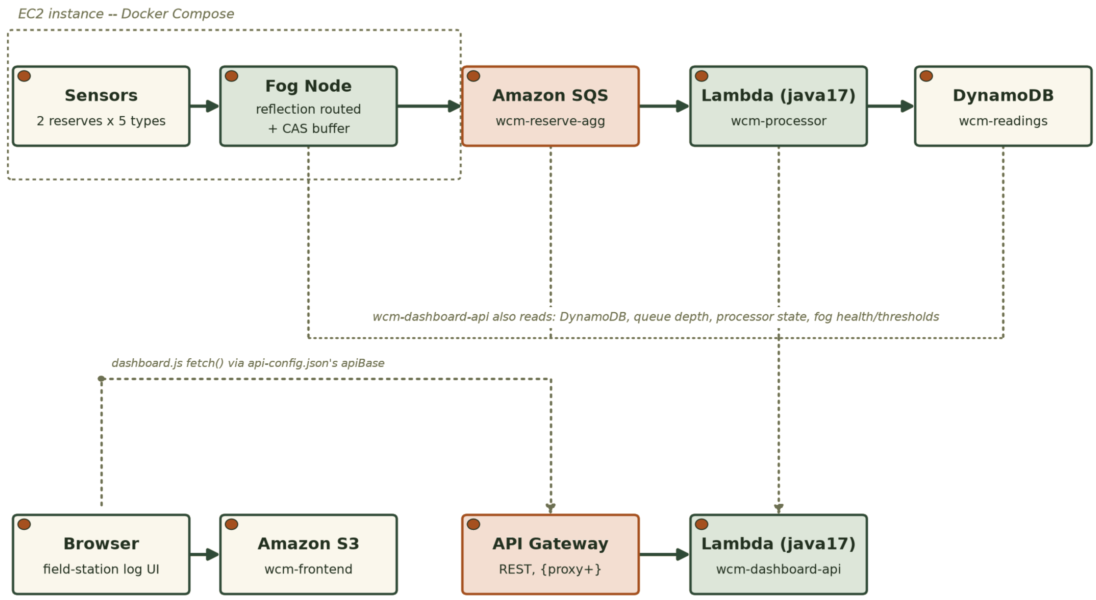

Fig. 1. Deployment topology: an ingest row (sensors through DynamoDB) and a query row (browser through the dashboard API), with the EC2-hosted sensor and fog tier the only piece not behind a managed AWS service.

and explicitly deferred rather than rejected outright. Lambda expects short, self-contained invocations with no state carried between them, which fits a queue consumer far better than it fits this buffer: the buffer's whole value comes from staying alive and accumulating readings between flushes, and turning that into a scheduled function would mean rebuilding the buffering logic around external storage instead of an in-process atomic reference. This project's cloud-deployment scope was always the backend layer, not the sensor and fog layer, so that rebuild is left for Section IV as future work rather than attempted here.

F. Security Considerations

Port 8000, reserved for the fog node, is the security group's sole inbound opening on this EC2 instance; nothing else gets in, including SSH, since the instance was never issued a key pair and all operator work instead happens over AWS Systems Manager Session Manager. That single open port exposes only read-only, non-sensitive status information, not data or control-plane access.

wcm-processor and wcm-dashboard-api both run under the Learner Lab account's one shared lab role because this account will not let a student mint a new IAM role, not because a single broad role was ever the intended design. Split apart, wcm-dashboard-api would only ever need read access to DynamoDB, to the SQS queue's attributes, and to Lambda metadata, while wcm-processor's entire footprint would shrink to permission to receive and delete queue messages and to write items, considerably tighter than what the shared role currently grants both functions by necessity. Leaving authentication off the dashboard's REST API was a deliberate choice, not an omission: every field it returns is an aggregated, non-personal reading from the reserve's sensors, so there is no sensitive payload for an authentication layer to be protecting in the first place.

III. Implementation

A. Software Components and Libraries

Java 17 runs across all four components (sensor, fog, processor, and dashboard API) with the plain Java HTTP server standing in for a framework like Spring, and the official AWS SDK for Java v2 handling every AWS call. The overall pipeline shape (sensors feeding a fog layer that windows, aggregates, and alerts before dispatching to a queue, a FaaS processor, a datastore, and a dashboard) follows standard, published designs cited throughout Section II. The implementation choices along each axis (the fog buffering mechanism, the alert-rule representation, the SQS publisher shape, the HTTP routing style, and the sensor-loop scheduling mechanism) were each made deliberately for this project, weighing the alternatives Section II discusses. The domain-specific code (sensor types, thresholds, the field-log logic, and the entire dashboard UI) is original to this project.

Jackson handles JSON throughout: parsing ingest payloads, rendering the thresholds response, and building the fog-to-SQS aggregate-plus-alerts message body. Chart.js draws the dashboard's waterhole-level trend chart client-side, vendored locally instead of pulled from a content-delivery network, so the page carries no third-party runtime dependency. Local development and continuous integration both run Docker Compose against LocalStack, an open-source AWS cloud emulator, so the same SQS, DynamoDB, and Lambda interactions exercised in production can be run day to day without touching AWS itself.

B. Testing Strategy

Each Maven module carries its own JUnit 5 test suite: 7 tests for sensors, 44 for the fog layer, 5 for the backend processor, and 26 for the dashboard, all currently passing. The gateway and router tests exercise the ingest endpoint and the reflection-driven route dispatch over a real Java HTTP server bound to an ephemeral port, not a unit test of validation logic in isolation. A 16-thread concurrent-ingest test proves the compare-and-swap retry loop never drops a reading under real contention. The thresholds-proxy tests cover both the success path and an unreachable-upstream path for the dashboard's fog-thresholds proxy.

Three tests specifically target the defects described in the next subsection. Two of them assert that a four-page fake DynamoDB scan (400, 400, 400, and 87 items) sums to exactly 1287 through the paginated item count, not just the first page's count. Another asserts that a 23-message flush window batches into sends of size ten, ten, and three, not twenty-three individual single-message sends.

C. Continuous Integration and Deployment

Every push and pull request triggers a two-stage GitHub Actions pipeline, not a single monolithic job. Stage one, unit-tests, executes the Maven suite across all four Java modules in complete isolation: no containers spun up, no socket ever opened. Stage two, integration, is gated on stage one succeeding and only then launches the LocalStack-backed stack, drives an end-to-end pipeline check through it, and tears the whole stack back down once finished, pass or fail, so no container is ever left running for the next job to trip over.

D. AWS Deployment and Defects Discovered

Nothing in this stack stayed a local simulation by the end: an AWS Academy Learner Lab account in us-east-1 now hosts every piece of it, sensor and fog containers included, alongside the queue, both Lambda functions, DynamoDB, API Gateway, and the S3 frontend, and that shift from emulator to a live account is exactly what turned up three problems Docker Compose and LocalStack alone never would have.

First, the row count reported by the dashboard's backend-statistics endpoint came from a single count-only scan. What that scan actually returns is not the table's true size but whatever fits in roughly 1 MB of scanned data; DynamoDB signals there is more by attaching a pagination cursor to the response, and unless the caller issues another scan starting from that cursor, the number reported stops short of reality with nothing to flag the shortfall. The bug is baked into how scan itself works, not something that only surfaces at a particular scale, and it was caught and corrected here before the table ever grew large enough to expose it visibly. The fix uses the SDK's own paginator, which follows the pagination cursor across pages automatically, rather than a hand-rolled loop or a recursive call.

Second, the fog node's flush cycle originally sent one SQS message per closed sensor-and-site group in a loop: functionally correct, but wasteful whenever more than one group closed in the same flush window, the normal case for a two-reserve, five-sensor-type deployment. The fix, a batch publisher, collects every window's aggregate into one list and chunks it at the ten-entry send limit, and the window cycle now calls it once per flush cycle instead of looping the earlier single-message send.

Third, a deploy script written specifically for deploying against LocalStack defaults to the LocalStack endpoint whenever the endpoint-override variable is unset, and unconditionally hardcodes a fixed test credential pair regardless of that variable's value. Left unguarded, this would have meant the script silently overwrote any real AWS credentials in the environment if it were ever pointed at a live account by mistake. This was not exercised in the actual deployment (the Lambda functions were created directly through the AWS CLI with their own explicit environment variables, entirely bypassing this script), but the risk was documented directly in the script with a comment clarifying it is LocalStack-only tooling, so a future reader does not assume it is safe to run against a live account.

None of the three fixes were taken on faith from the unit test suite; each was checked again against the running deployment. For the pagination and batching fixes, that meant the specific assertions already described (the four-page sum and the ten/ten/three chunk split) passing against the fakes stand in for what the actual DynamoDB and SQS clients do. For the dashboard, it meant loading the deployed page in an actual browser and reading its network requests directly: every static asset resolved, and the live data climbed across repeated reloads instead of sitting on a cached, stale count. Source: [GitHub repository URL].

TABLE II
AUTOMATED TEST SUITE BY MODULE

Module	Tests	Focus
sensors	7	bounded random walk, dual-rate config
fog (buffer+aggregate+alerts)	18	lock-free buffer, windowed aggregation, threshold rules
fog (publisher, routing, HTTP)	26	SQS batching, reflection routing, real-socket /ingest
backend/processor	5	DynamoDB item shape, sort-key logic
backend/dashboard	26	routes, pagination fix, real-socket + Lambda dispatch tests

E. Automated Test Coverage

Table II summarises the automated test suite as of the final verified deployment. All 82 tests pass against both the LocalStack-backed development environment and, for the parts of the suite that exercise genuine client-construction logic, against the fixes confirmed on the AWS deployment described in Section III-D.

F. Live Deployment Health

Independent verification did not stop at the deployed API's /api/health response, though that response was checked too and found consistent: gateway (the fog node's health check),

Fig. 2. The deployed Wildlife Conservation & Habitat Monitor dashboard, served from the S3 frontend during the live browser check: both reserves' gauge and field-station log panels, the four pipeline status indicators, and the waterhole-level trend chart.

queue, lambda, and pipeline every field true, the freshest reading under six seconds old. Each underlying AWS resource was then queried directly with the command-line tools, separately from anything the application itself reported: both Lambdas returned active with their last update successful, the event-source mapping for the queue feeding wcm-processor came back enabled, the EC2 host showed running and bound to its Elastic IP with a security group locked to inbound TCP 8000 only and no key pair attached, and the DynamoDB table returned active under on-demand billing.

Checking the backend-statistics item count twice, fifteen seconds apart, showed it climb from 362 to 378, evidence the pipeline was actively running, not serving a frozen or cached figure. The dashboard itself was opened in a real browser, not just probed with curl: the page, its stylesheet, its vendored chart library, and its dashboard script each came back with a 200 and no console errors, and both reserves' field-station log panels updated with live, changing data, correctly tripping alert flags against the threshold rules in Table I. Side by side for both reserves, the page shows a per-metric summary gauge for each of the five sensor types, a scrolling field-station log of recent windowed readings and any alerts they tripped, the four pipeline health indicators, and a waterhole-level trend comparing both sites, all refreshed by a client-side poll against the backend API every two and a half seconds rather than any push-based mechanism. Fig. 2 shows the deployed dashboard as it rendered during that browser check.

G. Cost Considerations

Every backend component other than the EC2 instance runs within AWS's request-based free-tier allowances at this project's data volume: DynamoDB on-demand billing charges per request rather than per provisioned throughput, both Lambda functions' invocation counts sit well inside the perpetual one-million-request Lambda free tier, and API Gateway and SQS are billed the same way, per request, with no idle cost between requests. The EC2 instance is the only continuously billed resource, since it runs the fog node and sensor containers as long-lived processes, not on-demand invocations. DynamoDB, Lambda, and API Gateway are what actually keep the dashboard and its API answering requests, so powering the EC2 instance off while idle does not break either one; what stops working is narrower than that: the health check's fog-reachability flag goes false, and no new readings land in the table until the instance comes back up.

H. Limitations

This evaluation has four honest gaps, named here rather than left for a reader to find unaided. The fog and sensor tier is the first: it still runs on one EC2 instance, so that specific tier keeps a single point of failure even though everything downstream of it is redundant AWS infrastructure; Section II-E's critical analysis explains why moving this tier to a fully serverless model was deferred, not attempted, but that does not make the resulting availability gap any less real. Second, this project ran inside an AWS Academy Learner Lab account, which is provisioned for bounded class sessions, not the kind of uninterrupted, multi-day run a production rollout would need to prove itself over, so every figure in this section reflects one continuous sitting, not sustained operation across days or weeks. Third, one broadly scoped IAM role is shared between both Lambda functions instead of each holding a least-privilege role, a constraint imposed by the Learner Lab account, discussed further in Section II-F, and not a deliberate security choice. Fourth, the automated integration suite verifies the pipeline against LocalStack, not the live AWS account itself, so the live evidence in this section rests on manual polling and browser inspection, not on an automated live-account test run.

IV. Conclusion

Wildlife habitat monitoring was the domain chosen to prove out a fog-to-serverless pipeline end to end: sensors and a fog layer with genuine configurability, a backend split across two Lambda functions rather than one, and an actual cloud deployment to stand behind the design choices made at every layer, not just a description of them on paper. By the close of this work all three layers existed as running, tested, deployed artefacts, each weighed against the alternatives that were realistically on the table, rather than a paper design left untested.

Two defects made it through the full local test suite and a complete LocalStack-backed integration run without tripping a single assertion: the DynamoDB pagination undercount, and the missing SQS batching. Neither one changes what a single-page scan or a single-message send returns at this project's small local data volume: they only change what gets silently dropped once a real account's table or a flush cycle's group count grows past that boundary, and no local test exercises data at that scale. What actually caught both was pointing the same code at the deployed AWS account and at the rendered dashboard in a real browser, not writing more local tests.

Application code was not where the actual difficulty sat. It was in the AWS Academy Learner Lab account's own platform and IAM restrictions, and in resisting the temptation to treat a green API check as evidence that the page a user actually sees was working too. Two extensions would matter most going forward: moving the fog and sensor tier off a continuously running EC2 instance onto a fully event-driven, scheduled-Lambda model, and, were the account's IAM restrictions ever lifted, splitting the shared lab role into two least-privilege roles, one for wcm-processor and one for wcm-dashboard-api.

REFERENCES

[1] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12)*, Helsinki, Finland, Aug. 2012, pp. 13–16.

[2] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[3] P. Prince, A. Hill, E. Piña Covarrubias, P. Doncaster, J. L. Snaddon, and A. Rogers, "Deploying acoustic detection algorithms on low-cost, open-source acoustic sensors for environmental monitoring," *Sensors*, vol. 19, no. 3, p. 553, 2019.

[4] M. M. Michael and M. L. Scott, "Simple, fast, and practical non-blocking and blocking concurrent queue algorithms," in *Proc. 15th Annu. ACM Symp. Principles of Distributed Computing (PODC '96)*, Philadelphia, PA, USA, May 1996, pp. 267–275.

[5] Y. Li, T. Tan, and J. Xue, "Understanding and analyzing Java reflection," *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 2, Art. 7, Feb. 2019.

[6] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, Stevenson, WA, USA, Oct. 2007, pp. 205–220.

[7] M. Shahrad *et al.*, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. 2020 USENIX Annu. Tech. Conf. (USENIX ATC '20)*, Jul. 2020, pp. 205–218.